



STOCKAHOLIC

Sistema de Gestão de Stock cm API REST



MAY 4, 2026

RELATÓRIO DE PROJETO FINAL
Afonso Batista e Rodrigo Vieira

Índice

1.	Introdução.....	2
2.	Arquitetura do Sistema	2
2.1	Visão geral.....	2
2.2	Diagrama de Arquitetura	3
2.3	Autenticação e Segurança	3
2.4	Estratégia de Cache	4
2.5	Estrutura do Repositório	4
3.	Estrutura da Base de Dados	5
3.1	Entidade Relação	5
3.2.	Relação entre tabelas.....	6
4.	Endpoints da API.....	7
4.1.	Autenticação.....	7
4.2.	Produtos	7
4.3.	Utilizadores.....	8
4.4.	Movimentos de Stock.....	8
4.5.	Categorias	8
4.6.	Imposter (Mock Externo)	9
5.	Exemplos de Pedidos à API	10
5.1.	Login (POST/auth/login)	10
5.2.	Listar Produtos (GET/produtos)	10
5.3.	Criar movimento(POST /movimentos).....	10
5.4.	Reset de Password(via Imposter).....	11
6.	Docker e Configuração	12
6.1	Modos de Execução	12
6.2	Iniciar em Modo Desenvolvimento.....	12
7.	Conclusão.....	13

1. Introdução

O Stockaholic é uma aplicação de gestão de stock desenvolvida no âmbito do Projeto Final da unidade curricular. A aplicação tem como objetivo permitir aos utilizadores gerir inventariários de produtos de forma simples e eficiente, registando entradas e saídas através de movimentos de stock.

O sistema segue uma arquitetura cliente-servidor, composta por uma API REST desenvolvida em ASP.NET Core 10 (.NET 10), um frontend web MVC também em .NET 10, uma base de dados PostgreSQL, cache distribuído com Redis e um serviço mock (imposter) para simulação de APIs externas.

Este relatório descreve a arquitetura do sistema, a estrutura da base de dados e os principais endpoints da API.

2. Arquitetura do Sistema

2.1 Visão geral

A arquitetura do Stockaholic segue o padrão de separação de preocupações, com camadas bem definidas:

- Frontend (Stockaholic.Frontend): Aplicação MVC ASP.NET Core 10 que consome a API e apresenta os dados ao utilizador através de views Razor.
- Backend / API REST (Stockaholic.Api): Exposta via HTTPS, responsável pela lógica de negócio, autenticação e acesso à base de dados.
- Base de Dados (PostgreSQL): Armazena de forma persistente todos os dados da aplicação.
- Cache Distribuído (Redis): Reduz a carga sobre a base de dados e acelera as respostas para operações de leitura frequentes.
- Imposter (Mountebank): Serviço mock que simula um sistema externo de reset de password.
- Docker / Docker Compose: Todo o ambiente pode ser levantado em modo de desenvolvimento ou produção com um único comando.

2.2 Diagrama de Arquitetura

O fluxo de pedidos segue a seguinte ordem:

- O utilizador acede ao Frontend (Stockaholic.Frontend) via browser.
- O Frontend envia pedidos HTTP à API REST (Stockaholic.Api).
- A API verifica primeiro a cache Redis; se não existir, consulta o PostgreSQL.
- Para operações de reset de password, a API comunica com o Imposter (Mountebank na porta 3000).
- A API devolve a resposta JSON ao Frontend, que renderiza a view correspondente.

Componente	Tecnologia	Função
Frontend	ASP.NET Core 10 MVC	Interface web para o utilizador final
API REST	ASP.NET Core 10 Web API	Lógica de negócio, autenticação JWT, endpoints REST
Base de dados	PostgreSQL	Persistência de dados (EF Core)
Cache	Redis (StackExchange.Redis)	Cache distribuído com TTL de 5 minutos
Imposter(Mock)	Mountebank	Simulação de serviço externo de reset de password
Containerização	Docker + Docker Compose	Orquestração de serviços (dev e produção)

2.3 Autenticação e Segurança

A API utiliza JWT (JSON Web Tokens) para autenticação. Todos os endpoints, à exceção do login e forgot-password, requerem o envio do token no header Authorization: Bearer <token>.

As passwords dos utilizadores são armazenadas de forma segura usando PBKDF2 com SHA-256, salt aleatório de 16 bytes e 10.000 iterações. O ClockSkew é definido como zero, não havendo período de graça na expiração do token.

2.4 Estratégia de Cache

O serviço de cache (CacheService) implementa o padrão cache-aside com Redis, com um TTL (Time To Live) padrão de 5 minutos. Sempre que um recurso é criado, atualizado ou eliminado, as entradas de cache correspondentes são invalidadas (cache eviction).

GET /produtos → cache key: produtos:all
GET /produtos/{id} → cache key: produtos:{id}
GET /movimentos → cache key: movimentos:all
GET /utilizadores → cache key: utilizadores:all

2.5 Estrutura do Repositório

```
stockaholic/  
├── Stockaholic.Api/           # API REST (.NET 8)  
│   ├── Auth/                 # TokenService (JWT)  
│   ├── Cache/                 # CacheService (Redis)  
│   ├── Controllers/          # Endpoints REST  
│   ├── Data/                  # DbContext (EF Core)  
│   └── Models/                # Entidades  
├── Stockaholic.Frontend/      # Frontend MVC  
├── imposters/                 # Mountebank config (imposter.json)  
├── scripts/                   # SQL de inicialização (postgres_init.sql)  
├── docker-compose.dev.yml     # Compose desenvolvimento  
└── docker-compose.prod.yml    # Compose produção
```

3. Estrutura da Base de Dados

3.1 Entidade Relação

A base de dados PostgreSQL é inicializada automaticamente pelo script `postgres_init.sql` e gerida via Entity Framework Core. São quatro tabelas principais:

Tabela Categorias

Coluna	Tipo	Restrições	Descrição
id	INTEGER (IDENTITY)	PK, NOT NULL	Identificador único da categoria
nome	VARCHAR(255)	NOT NULL	Nome da categoria

Tabela Produtos

Coluna	Tipo	Restrições	Descrição
id	INTEGER (IDENTITY)	PK, NOT NULL	Identificador único da Produto
nome	VARCHAR(255)	NOT NULL	Nome da Produto
categoria_id	INTEGER	FK -> categorias(id)	Categoria a qual o produto pertence
preço	DECIMAL(10,2)	NOT NULL	Preço unitário do Produto

Tabela Utilizadores

Coluna	Tipo	Restrições	Descrição
id	INTEGER (IDENTITY)	PK, NOT NULL	Identificador único da Produto
nome	VARCHAR(255)	NOT NULL	Nome Completo
email	VARCHAR(320)	NOT NULL, UNIQUE	Identificador único da Produto
password_hash	TEXT	NOT NULL	Hash PBKDF2-SHA256 da password
Password_salt	TEXT	NOT NULL	Salt aleatório (16 bytes, Base64)

Tabela Movimentos

Coluna	Tipo	Restrições	Descrição
id	INTEGER (IDENTITY)	PK, NOT NULL	Identificador único
nome	VARCHAR(255)	NOT NULL	Nome descritivo do movimento
timestamp	TIMESTAMPTZ	NOT NULL	Data/hora do movimento (com fuso)
produto_id	INTEGER	FK → produtos(id)	Produto afetado
delta	INTEGER	NOT NULL	Variação de stock (+/-)
descricao	TEXT	NULL	Observações adicionais
utilizador_id	INTEGER	FK → utilizadores(id)	Utilizador Responsável

3.2. Relação entre tabelas

- produtos.categoria_id → categorias.id (ON UPDATE CASCADE, ON DELETE RESTRICT)
- movimentos.produto_id → produtos.id (ON UPDATE CASCADE, ON DELETE RESTRICT)
- movimentos.utilizador_id → utilizadores.id (ON UPDATE CASCADE, ON DELETE RESTRICT)

A opção ON DELETE RESTRICT garante integridade referencial: um produto com movimentos registados não pode ser eliminado sem primeiro remover os seus movimentos associados.

4. Endpoints da API

A API é documentada via Swagger/OpenAPI, acessível em <http://localhost:8080/swagger>. Todos os endpoints requerem autenticação JWT, exceto os de login e forgot-password.

4.1. Autenticação

Método	EndPoint	Auth	Descrição
POST	/auth/login	Público	Autentica utilizador e retorna token JWT
POST	/auth/forgot-password	Público	Comunica com o imposter para reset de password
POST	/auth/me	JWT	Retorna informação do utilizador autenticado

4.2. Produtos

Método	EndPoint	Auth	Descrição
GET	/produtos	JWT	Lista todos os produtos (com cache Redis 5 min)
GET	/produtos/{id}	JWT	Obtém produto por ID (com cache Redis)
GET	/produtos/numero	JWT	Contagem total de produtos distintos
GET	/produtos/valor	JWT	Valor total do stock (preço × quantidade)
POST	/produtos	JWT	Cria novo produto e invalida cache
PUT	/produtos/{id}	JWT	Atualiza produto completo e invalida cache
PATCH	/produtos/{id}	JWT	Atualiza campos parciais do produto
DELETE	/produtos/{id}	JWT	Elimina produto e invalida cache

4.3. Utilizadores

Método	EndPoint	Auth	Descrição
GET	/utilizadores	JWT	Lista todos os utilizadores (com cache Redis)
GET	/utilizadores/{id}	JWT	Obtém utilizador por ID (com cache Redis)
POST	/utilizadores	JWT	Cria novo utilizador(hash PBKDF2 + salt)
PUT	/utilizadores/{id}	JWT	Substitui dados do utilizador
PATCH	/utilizadores/{id}	JWT	Atualiza campos parciais (nome/email/password)
DELETE	/utilizadores/{id}	JWT	Elimina utilizador e invalida cache

4.4. Movimentos de Stock

Método	EndPoint	Auth	Descrição
GET	/movimentos	JWT	Lista todos os movimentos (com cache Redis)
GET	/movimentos/{id}	JWT	Obtém movimento por ID
GET	/movimentos/recentes	JWT	Lista movimentos mais recentes (ordenados por timestamp desc)
GET	/movimentos/stock	JWT	Agrega stock atual por produto (soma de deltas)
GET	/movimentos/topvalor	JWT	Top produtos por valor de stock
POST	/movimentos	JWT	Regista novo movimento (+/-) e invalida cache
DELETE	/movimentos/{id}	JWT	Elimina movimento e invalida cache

4.5. Categorias

Método	EndPoint	Auth	Descrição
GET	/categorias	JWT	Lista todas as categorias (com cache Redis)
GET	/categorias/{id}	JWT	Obtém categoria por ID
POST	/categorias	JWT	Cria nova categoria
PUT	/categorias/{id}	JWT	Atualiza categoria completa
DELETE	/categorias/{id}	JWT	Elimina categoria (se sem produtos associados)

4.6. Imposter (Mock Externo)

O imposter é um serviço Mountebank que corre na porta 3000 e simula um sistema externo de reset de password. É invocado pela API quando o utilizador solicita a recuperação da password.

Método	Endpoint (porta 3000)	Auth	Descrição
POST	/reset-password	Público	Retorna URL de reset simulada com token fictício

Exemplo de resposta do Imposter:

```
{ "message": "Reset link generated",  
  "resetUrl": "https://fake-reset.local/reset?token=abc123" }
```

5. Exemplos de Pedidos à API

5.1. Login (POST/auth/login)

Request Body:
POST /auth/login Content-Type: application/json { "email": "admin@admin.com", "password": "123" }
Response(200 OK)
{ "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..." }

5.2. Listar Produtos (GET/produtos)

Request Body:
GET /produtos Authorization: Bearer <token>
Response(200 OK)
{ "id": 1, "nome": "Caneta Azul", "categoriaId": 2, "preco": 0.99 }, { "id": 2, "nome": "Caderno A4", "categoriaId": 2, "preco": 3.50 }

5.3. Criar movimento(POST /movimentos)

Request Body:
POST /movimentos Authorization: Bearer <token> Content-Type: application/json { "nome": "Entrada de stock", "produtoId": 1, "delta": 50, "descricao": "Receção de fornecedor", "utilizadorId": 1 }
Response(201 Created)
{ "id": 15, "nome": "Entrada de stock", "produtoId": 1, "delta": 50, "utilizadorId": 1, "timestamp": "2026-05-03T18:30:00Z" }

5.4. Reset de Password(via Imposter)

Request Body:
POST /auth/forgot-password Content-Type: application/json { "email": "utilizador@exemplo.com" }
A API comunica internamente com o Mountebank (porta 3000) e devolve a resposta:
 { "message": "Reset link generated", "resetUrl": "https://fake-reset.local/reset?token=abc123" }

6. Docker e Configuração

6.1 Modos de Execução

O projeto inclui dois ficheiros Docker Compose:

- `docker-compose.dev.yml`: Levanta apenas o PostgreSQL e o Mountebank. A API e o Frontend correm localmente com `dotnet run`.
- `docker-compose.prod.yml`: Levanta todos os serviços em containers (API, Frontend, PostgreSQL, Redis, Mountebank).

6.2 Iniciar em Modo Desenvolvimento

Iniciar base de dados e imposter
<code>docker compose -f docker-compose.dev.yml up -d</code>
Correr a API
<code>dotnet run --project Stockaholic.Api/Stockaholic.Api.csproj</code>
Correr o Frontend
<code>dotnet run --project Stockaholic.Frontend/Stockaholic.Frontend.csproj</code>

Após iniciar, a Swagger UI está disponível em `http://localhost:8080/swagger` e o frontend em `http://localhost:5018`. As credenciais iniciais são `admin@admin.com / 123`.

7. Conclusão

O Stockaholic cumpre os requisitos definidos no enunciado do projeto, implementando:

- API REST em ASP.NET Core 10 com endpoints CRUD completos para todas as entidades.
- Autenticação segura via JWT com armazenamento de passwords usando PBKDF2-SHA256.
- Cache distribuído com Redis integrado via StackExchange.Redis, com estratégia cache-aside e invalidação automática.
- Base de dados PostgreSQL com esquema relacional normalizado e restrições de integridade referencial.
- Serviço mock (Mountebank) para simulação de sistema externo de reset de password.
- Frontend MVC que consome a API e apresenta uma interface completa para gestão de stock.
- Docker Compose para orquestração de todos os serviços em modo de desenvolvimento e produção.
- Documentação automática da API via Swagger/OpenAPI.

O projeto demonstra a aplicação prática de padrões modernos de desenvolvimento web, incluindo separação de preocupações, segurança por design, caching para desempenho e containerização para portabilidade.